Server side scripting with database connectivity

Unit-2

Prepared by Laxman pr Pant.

Introduction to server side scripting language

- **Server-side scripting:** is a technique used in web development which involves employing scripts on a web server which produces a response customized for each user's (client's) request to the website.
- Server-side scripting is often used to provide a customized interface for the user.
- It can also be used to provide dynamic websites.
- It helps work with the back end.
- It doesn’t depend on the client.
- It runs on the web server.
- It helps provide a response to every request that comes in from the user/client.
- This is not visible to the client side of the application.
- It requires the interaction with the server for the data to be process.
- Server side scripting requires languages such as PHP, ASP.net, ColdFusion, Python, Ruby on Rails.

Client-side Scripting

- It helps work with the front end.
- It is visible to the users.
- The scripts are run on the client browser.
- It runs on the user/client's computer.
- It depends on the browser's version.
- It doesn't interact with the server to process data.
- Client side scripting involves languages such as HTML, CSS, JavaScript.
- It helps reduce the load on the server.
- It is considered to be less secure in comparison to client side scripting

- There are various type server but in our web application ,mainly used server are:

Application server

- The **application server** is a framework, an environment where applications can run, no matter what they are or what functions they perform
- There are a number of different types of application servers, including Java, PHP and .NET Framework application servers.

Web server

- Computer or collection of computers used to deliver web pages and other content to multiple users.
- Eg. Apache HTTP Server, Apache Tomcat, Microsoft IIS

Database server

- A **database server** is a computer system that provides other computers with services related to accessing and retrieving data from a database.
- Ex. Mysql, sql,oracle etc

What is PHP?

- PHP is an acronym for "PHP: Hypertext Preprocessor"
- PHP is an open-source, interpreted, and object-oriented scripting language that can be executed at the server-side
- PHP is a widely-used, open source scripting language
- PHP scripts are executed on the server
- It is powerful enough to be at the core of the biggest blogging system on the web (WordPress)!

Why PHP?

- Easy to learn.
- Best suited for rapid application development.
- Easy to implement.
- PHP community is big support for the programmers.
- Runs on almost every platform.
- Its free and develop dynamic web pages
- PHP is most used server side programming language.

PHP Basics Syntax

- PHP files have a file extension of ".php".

- PHP file may contain

  - HTML(Everything is HTML code in the file by default).
  - CSS(code in style tag)
  - JavaScript(code in Script tag)
  - PHP (code in **<?php … ?>**)

  PHP script can be used any number of times in the php file.

  Every piece of PHP script in the page is enclosed in <?php … ?>

Comments in PHP

single-line comments: Single-line comments allow narrative on only one line at a time. For example:

```
<!DOCTYPE html>
<html>
<body>
<?pup
// This is a single-line comment
echo (" Above given // is  a single-line comment");
?>
</body>
</html>
```

Multi-line comments in PHP

Mulit-line comments: Multi-line comments in PHP allows to write multi at a time. For example:

```
<!DOCTYPE html>
<html>
<body>
<?php
/*
This is a multiple-lines comment block
that spans over multiple lines
*/
?>
</body>
</html>
```

PHP variables

- Variables are used to store constants.
- These constants could be string, numbers or arrays.
- When a variable is declared, it can be used over and over again in your script.
- All variables in PHP start with a $ sign symbol

Example:

```
<?php
$txt=“computer”;
$x=4;
?>
```

- In PHP, the variable is declared automatically when you use it.

PHP Data Types

PHP supports the following data types:

- String
- Integer
- Float (floating point numbers- also called double)
- Boolean
- Array
- Object
- Null
- Resource

PHP variable scope

- The scope of a variable is defined as its range in the program under which it can be accessed
- PHP has four different variable scopes
  - **Local**
  - **Global**
  - **Static**
  - **Parameter**

Local variable:

- The variables that are declared within a function are called local variables for that function

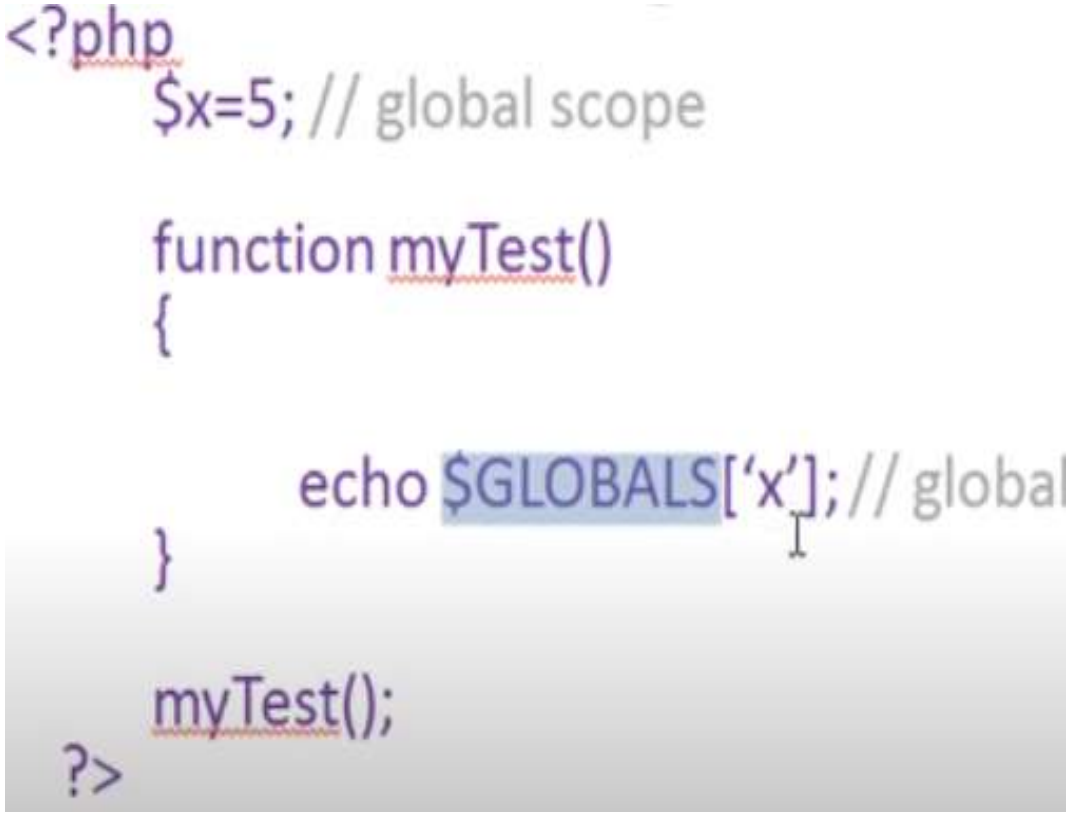
Example:

```
<?php
   function local_var()
   {
      $num = 45;  //local variable
      echo "Local variable declared inside the function is: ". $num;
   }
   local_var();
?>
```

Global variable

- The global variables are the variables that are declared outside the function
- These variables can be accessed anywhere in the program.
- To access the global variable within a function, use the GLOBAL keyword before the variable. However, these variables can be directly accessed or used outside the function without any keyword.
- Example:

```
<?php
  $x=5; //global scope
  function myTest()
  {
      global $x;
      echo $x;  //global scope
  }
myTest();
?>
```



Static variable

- When a function is completed, all of its variable are normally deleted.
- Sometimes you want a local variable not to be deleted.
- To do this, use the **Static** keyword when you first declare the variable.

Example:

```
<?php
  function myTest()
  {
      static $x=0;
      echo $x;
      $x++;
  }
  myTest()
  myTest()
  myTest()
?>
```

Parameter scope:

- A parameter is a local variable whose value is passed to the function by the calling code.
- Parameters are declared in a parameter list as part of the function declaration.

Example

```
<?php
  function myTest($x)
  {
      echo $x;
  }
  myTest(5);
?>
```

PHP operators

- PHP operator s can be categorized in the following group.
  - Arithmetic operator
  - Assignment Operator
  - Comparison Operator
  - Increment/Decrement Operator
  - Logical Operator
  - String Operator
  - Array Operator

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y
*	Multiplication	$x * $y	Product of $x and $y
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y

Assignment Operators

Assignment	Same as...	Description
x = y	x = y	The left operand gets set to the value of the expression on the right
x += y	x = x + y	Addition
x -= y	x = x – y	Subtraction
x *= y	x = x * y	Multiplication
x /= y	x = x / y	Division
x %= y	x = x % y	Modulus

Comparison Operators

Operator	Name	Example	Result
==	Equal	$x == $y	Returns true if $x is equal to $y
===	Identical	$x === $y	Returns true if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	Returns true if $x is not equal to $y
<>	Not equal	$x <> $y	Returns true if $x is not equal to $y
!==	Not identical	$x !== $y	Returns true if $x is not equal to $y, or they are not of the same type
>	Greater than	$x > $y	Returns true if $x is greater than $y
<	Less than	$x < $y	Returns true if $x is less than $y
>=	Greater than or equal to	$x >= $y	Returns true if $x is greater than or equal to $y
<=	Less than or equal to	$x <= $y	Returns true if $x is less than or equal to $y
<=>	Spaceship	$x <=> $y	Returns an integer less than, equal to, or greater than zero, depending on if $x is less than, equal to, or greater than $y. Introduced in PHP 7.

Increment/decrement Operators

Operator	Name	Description
++$x	Pre-increment	Increments $x by one, then returns $x
$x++	Post-increment	Returns $x, then increments $x by one
--$x	Pre-decrement	Decrements $x by one, then returns $x
$x--	Post-decrement	Returns $x, then decrements $x by one

Logical Operators

Operator	Name	Example	Result
and	And	$x and $y	True if both $x and $y are true
or	Or	$x or $y	True if either $x or $y is true
&&	And	$x && $y	True if both $x and $y are true
||	Or	$x || $y	True if either $x or $y is true
!	Not	!$x	True if $x is not true

String Operators

Operator	Name	Example	Result
.	Concatenation	$txt1 . $txt2	Concatenation of $txt1 and $txt2
.=	Concatenation assignment	$txt1 .= $txt2	Appends $txt2 to $txt1

Control Structures in PHP

- **Branching statement**
  - If statement
  - If … else statement
  - If …else if… else statement
  - Nested if…else statement
  - Switch statement
- **Looping statement**
  - While loop
  - Do…while loop
  - For loop
  - Foreach loop

If statement in PHP

➢The if statement executes some code if one condition is true.

Syntax

if (*condition*)

{

code to be executed if condition is true;

}

For Example:

```
<?php
$day = 18;
if ($day < "20") {
  echo "Have a good day!";
}
?>
```

If ….else statement in PHP

- The if...else statement executes some code if a condition is true and another code if that condition is false.

Syntax:

```
if (condition)
{
  code to be executed if condition is true;
}
else
{
  code to be executed if condition is false;
}
```

For Example:

```
<?php
$t = date("H");

if ($t < "20")
{
  echo "Have a good day!";
}
else
 {
  echo "Have a good night!";
}
?>
```

IF…elseif…..else statement

- The if...elseif...else statement executes different codes for more than two conditions.

Syntax:

```
if (condition)
{
  code to be executed if this condition is true;
}
elseif (condition)
{
  code to be executed if first condition is false and this condition is true;
}
else
{
  code to be executed if all conditions are false;
}
```

For Example:

```
<!DOCTYPE html>
<html>
<body>
<?php
$t = date("H");
if ($t < "10") {
  echo "Have a good morning!";
} elseif ($t < "20") {
  echo "Have a good day!";
} else {
  echo "Have a good night!";
}
?>
</body>
</html>
```

Switch Case Statement in PHP:

- The switch statement is used to perform different actions based on different conditions. The PHP switch Statement use the switch statement to **select one of many blocks of code to be executed**.

Syntax:

```
switch (n) {
  case label1:
    code to be executed if n=label1;
    break;
  case label2:
    code to be executed if n=label2;
    break;
  case label3:
    code to be executed if n=label3;
    break;
    ..............n case
  default:
    code to be executed if n is different from all labels;
}
```

For Example:

```
<?php
$favcolor = "red";
switch ($favcolor) {
  case "red":
    echo "Your favorite color is red!";
    break;
  case "blue":
    echo "Your favorite color is blue!";
    break;
  case "green":
    echo "Your favorite color is green!";
    break;
  default:
    echo "Your favorite color is neither red, blue, nor green!";
}
?>
```

Some Project Questions

1) WAP in PHP to input any number and checks whether the given number is odd or even.
2) WAP in PHP to input any number and checks whether the given number is positive or negative.
3) WAP in PHP to input any five subjects marks . Find the average marks of them and check the division of students where assume division yourself like if percentage is >=90 then he/she has got distinction.
4) WAP in PHP to input three numbers and display the largest number among them.
5) Explain the switch case statement with example.
6) Explain the conditional /decision making statements in pup.

Looping statements in PHP

- Loops are used to execute the same block of code again and again, as long as a certain condition is true.
- In PHP, we have the following loop types:
- **while -** loops through a block of code as long as the specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as the specified condition is true
- **for -** loops through a block of code a specified number of times
- **foreach -** loops through a block of code for each element in an array

The PHP while Loop

- The while loop - Loops through a block of code as long as the specified condition is true.

Syntax :

while (*condition is true*)

{
  code to be executed;
}

Example

//The example below displays the numbers from 1 to 5:

```
<?php
$x = 1;

while($x <= 5) {
  echo "The number is: $x <br>";
  $x++;
}
?>
```

The PHP do…while Loop

- The do...while loop - Loops through a block of code once, and then repeats the loop as long as the specified condition is true.
- The do...while loop will always execute the block of code once, it will then check the condition, and repeat the loop while the specified condition is true.

Syntax:

```
do {
  code to be executed;
} while (condition is true);
```

The PHP do...while Loop

Example:

```
<?php
$x = 1;

do {
  echo "The number is: $x <br>";
  $x++;
} while ($x <= 5);
?>
```

Output

The number is: 1
The number is: 2
The number is: 3
The number is: 4
The number is: 5

The PHP for Loop

- The for loop - Loops through a block of code a specified number of times.
- The for loop is used when you know in advance how many times the script should run.

Syntax:

```
for (init counter; test counter; increment counter) {
  code to be executed for each iteration;
}
```

Example:

```
<?php
for ($x = 0; $x <= 10; $x++) {
  echo "The number is: $x <br>";
}
?>
```

Output

The number is: 1
The number is: 2
The number is: 3
The number is: 4
The number is: 5

The PHP for each Loop or foreach () array method

- The foreach loop - Loops through a block of code for each element in an array.
- The foreach loop works only on arrays, and is used to loop through each key/value pair in an array.

Syntax:

```
forEach (function);

  code to be executed;
}
```

The PHP foreach Loop

Example:

```
<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $value) {
  echo "$value <br>";
}
?>
```

Output

red

green

blue

yellow

The PHP Break and Continue statement.

PHP Break:

- We have already seen the break statement used in an earlier topic[switch case] . It was used to "jump out" of a switch statement.
- The break statement can also be used to jump out of a loop.

Example:

```
<?php
for ($x = 0; $x < 10; $x++) {
  if ($x == 4) {
    break;
  }
  echo "The number is: $x <br>";
}
?>
```

Output

The number is: 0
The number is: 1
The number is: 2
The number is: 3

The PHP Break and Continue statement.

PHP Continue Statement

- The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Example

- ```
  <?php
  for ($x = 0; $x < 10; $x++) {
    if ($x == 4) {
      continue;
    }
    echo "The number is: $x <br>";
  }
  ?>
  ```

Output

```
The number is: 0
The number is: 1
The number is: 2
The number is: 3
The number is: 5
The number is: 6
The number is: 7
The number is: 8
The number is: 9
```
```

Some looping questions

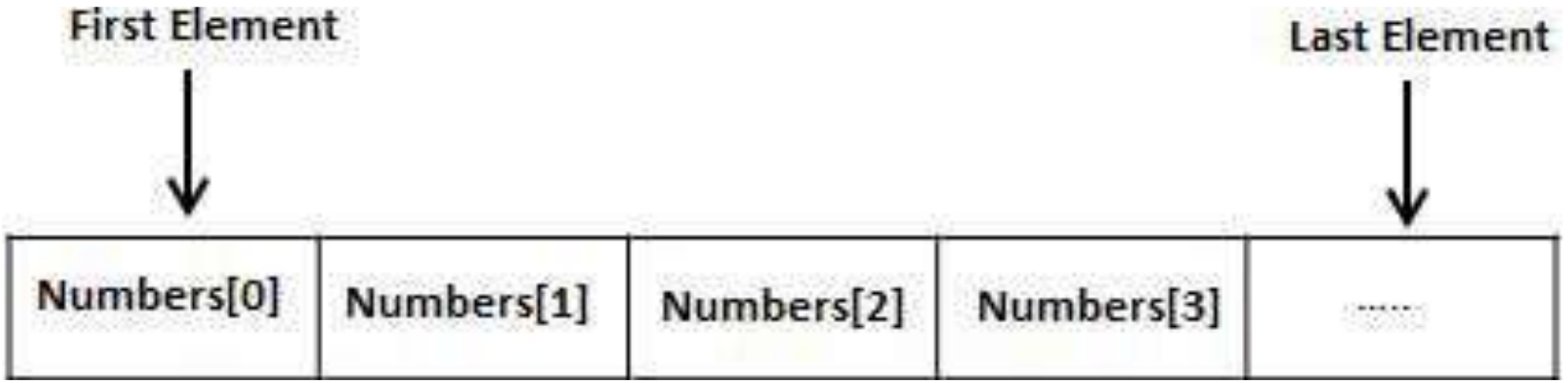
- Write a program in PHP to print the first 10 natural numbers.
- Write a program in PHP to print the first 20 even numbers.
- Write a program in PHP to print the first 13 odd numbers.
- Write a program to input any number and check weather the given number is prime or not.
- Write a program to input to print the Fibonacci series like 0 1 1 2 3 5……..10 th term.
- Write a program to display the sum of 1+2+3+4…….+100.
- Write a program to display the sum of 20 even numbers.
- Explain break and continue statements with an example.

Array in PHP

- Array is linear collection of similar elements.
- It is used to hold multiple values of similar type in a single variable.
- Each element in the array has its own index so that it ca be easily accessed.

Syntax:

$<array_name>= array(<array_elements>);



Types of Array

- There are three types of array in php.
    1. Indexed array or numeric array
    2. Associative Array
    3. Multidimensional array

Indexed or Numeric array

- These array can store numbers, strings and any object but their index will be represented by numbers.
- By default array index starts from zero.

Example:

```
$x=array("Ram","Shyam","Geeta","Sita");
foreach($x as $value){
echo $value. "<br>";
}
```

Here,

$x[0] indicates "Ram"

$x[1] indicates "Shyam"

$x[2] indicates "Geeta"

$x[3] indicates "Sita"

Associative Array:

- The associative arrays are very similar to numeric arrays in terms of functionality but they are different in terms of their index.
- Associative array will have their **index as String** so that you can establish a strong association between key and values.
- An associative array each ID key is associated with a value.

Syntax:

$<array_name>=array(<key1=>val1>, <key2=>val2>, … <keyn=>valn>);

Example:

```
$age=array(“amit”=>20, ”rahul”=>31, “tarun”=>25);
foreach($age as $key=>$value){
echo $key=$value. “<br>”;
}
```

Here, $age[‘amit’] =20;

$age[‘rahul’]=31;

$age[‘tarun’]=25;

Multidimensional Array:

- PHP multidimensional array is also known as array of arrays.
- It allows you to store tabular data in an array.
- PHP multidimensional array can be represented in the form of matrix which is represented by row * column.

Syntax:

$<array_name>=array(array<elements>,array<elements>….array<elements>);

- Eg: $num=array(array(1,2,3), array(4,5,6), array(7,8,9));

  $num[0][0]=1

  $num[0][1]=2

  ……

  $num[2][2]=9

```php
<?php
$b=array(array(9,9,9,7,4),array(9,8,4,2,3));
echo "<pre>";
echo print_r($b);
echo"</pre>";
echo"<hr>";
for($i=0;$i<2;$i++)
{
        for($j=0;$j<5;$j++)
        {
                echo"The number is=";
                echo $b[$i][$j];

                echo"<br>";

        }
}
?>
```

Exercise

- **Write a PHP program to create a multidimensional array that holds the marks of three subject like math Nepali and physics of three student Ram,Shyam,Hari. Then display the average marks of each student.**
- **Write a PHP program to input 30 students marks in a subject and count the pass student's where the pass marks is >=32**
- **Write a PHP program to input 10 employees salary and display the employees salary where the salary is >=32000 by using associative array.**

Exercise

Write a PHP program to create a multidimensional array that holds the marks of three subject like math Nepali and physics of three student Ram,Shyam,Hari. Then display the total and average marks of each student.

```
<?php

//Create a multidimensional array to store marks

$marks = array( "Ram" => array("math" => 85, "Nepali" => 90, "physics" => 78), "Shyam" => array("math" => 75, "Nepali" => 88, "physics" => 92), "Hari" => array("math" => 92, "Nepali" => 78, "physics" => 85) );

  foreach ($marks as $student => $subjectMarks)

{

$totalMarks = array_sum($subjectMarks);

$averageMarks = $totalMarks / count($subjectMarks);

echo "$student's average marks: $averageMarks\n";

}

 ?>
```

Function in php

- PHP function is a piece of code that can be reused many times.
- It can take input as argument list and return value.
- **Syntax for defining function**

```
<?php
function  <function name>(<parameters>)
{
        //statements;
}
?>
```

- There are two types of function
  - User-defined function
  - Built- in function

User defined Functions

- Functions those are defined by user according to user requirements.
- To create a new function, use the **function** keyword

Syntax:

```
Function <function_name>(<parameters>)
{
    //code to be executed;
    //return statement;
}
```

Note:

- A function should start with keyword function and all the function code should be put inside { and} brace
- A function name can start with a letter or underscore not a number
- The function names are case-insensitive**.**
- **Return statements and parameters are optional.**

Eg: a simple function that writes my name when it is called.

```
<?php
  function printName()
  {
     echo(“Laxman");
  }
  echo("My name is ");
  printName();
?>
```

Output: My name is Laxman.

Return values

- A function call can be anywhere in the program, and it can be inside another function too.
- A simple function that returns value when it is called:

```
<?php
 function add($x,$y)
  {
     $total=$x+$y;
     return $total;
  }
   echo "1 + 16 = " . add(1,16);
?>
```

Output: 1 + 16 = 17

- Note: Function will return only one value.

Function with arguments:

- Information may be passed to functions via the argument list, which is a comma-delimited list of expressions.

```
<?php
 function Multiply($x,$y)
 {
    $total=$x*$y;
    echo("The Multiplication of two number=$total");
 }
 $a=6;$b=9;
 Multiply($a,$b);
 ?>
```

Default Parameters

- Default parameters enable you to specify a default value for function parameters that aren’t passed to the function during the function call.
- The default values you specify must be a constant value and default value can be specified for each of the last arguments.
- To do this, simply use the assignment operator and a value for the arguments in the function definition.
- If the value is specified then this default value is ignored and the passed value is used instead.

Example

```
<?php
echo "<h3>Use of Default Parameter<br></h3>";
function increment($num, $increment = 1)
{
$num += $increment;
echo $num."<br>";
}
$num = 4;
increment($num);
increment($num, 3);
?>
```

Output: 5

7

Pass by value and pass by reference

- Passing a variable by reference or by value to a function is a very useful concept for PHP programmers.
- Normally, when you are passing a variable to a function , you are passing “by value” which means the PHP processor will make a duplicate variable to work on.
- When you pass a variable to a function “by reference” you passing the actual variable and all work done on that variable will be done directly on it.

Pass by value-Example

```
<?php
        $a=10; $b=20;
        echo "<h3>Before Swapping<br></h3>";
        echo "a:".$a."b:".$b."<br>";
        swap($a,$b);
        echo "<h3>After Swapping<br></h3>";
        echo "a:".$a."b:".$b."<br>";
        function swap($no1,$no2)
        {
          $temp=$no1;
          $no1=$no2;
           $no2=$temp;
        }
?>
```

Output:

Before Swapping

a:10 b:20

After Swapping

a:10 b:20

Pass by reference-Example

```
<?php
        $a=10; $b=20;
        echo "<h3>Before Swapping<br></h3>";
        echo "a:".$a."b:".$b."<br>";
        swap($a,$b);
        echo "<h3>After Swapping<br></h3>";
        echo "a:".$a."b:".$b."<br>";
        function swap(&$no1,&$no2)
        {
          $temp=$no1;
          $no1=$no2;
           $no2=$temp;
        }
?>
```

Output:

Before Swapping

a:10 b:20

After Swapping

a:20 b:10

Recursive Function

- Recursive function is a function which calls itself again and again until the termination condition arrive.

```
<?php
function fact($n)
{
if ($n === 0)
{
// our base case
return 1;
}
else
{
return $n * fact($n-1); // <--calling itself.
}
}
echo fact(5);
?>
```

Built-in Functions in PHP

- are predefined functions in PHP
- Built in functions are functions that exist in PHP installation package.
- The built in functions can be classified into many categories
- **String Functions**
  - **Trim() :**Remove whitespace or other characters from the beginning and end of the string.
  - **strcasecmp():**It is used to compare two strings.
  - **Strlen() :**It is used to return the length of a string.
  - **Strrev():** It is used to reverse a string
  - **Strtolower():** Convert the string in lowercase
  - **Strtoupper():** Convert the strings in uppercase
  - **ucwords():** Make the first character of each word in a string to uppercase
  - **explode():** It is used to break a string into an array.
  - **implode(): :** It is used to return a string from the elements of an array

Numeric Functions

- Sqrt(): Returns the square root of a number.
- Round(): Round off a number with decimal points to the nearest whole number.
- Abs(): Returns the absolute value of a number.
- Cos () : returns cosine
- Sin () : returns sine
- Tan() returns tangent

Array Functions:

- **count()-** counts no. of elements in an array.
- **array_reverse()-** returns an array in the reverse order
- **array_merge()-** Merges one or more arrays into one array.
- **array_pop()-** Deletes the last element of an array.
- **array_push()-** Inserts one or more elements to the end of an array.
- **array_search()-** Searches an array for a given value and returns the key.
- **array_slice()-** Returns selected parts of an array.
- **array_splice()-** Removes and replaces specified elements of an array.
- **sort()-** sorts an array(only for numeric family in ascending order).
- **asort()-** Sorts an associative array in ascending order, according to the value.
- **ksort()-** Sorts an associative array in ascending order, according to the key.
- **rsort()-** Sorts an indexed array in descending order.
- **arsort()-** sort associative arrays in descending order, according to the value
- **Krsort()-** sort associative arrays in descending order, according to the key

Superglobals

- Several predefined variables in PHP are “superglobals”, which means they are available in all scopes throughout a script.There is no need to invoke **global $variable;**
- Some of the superglobals variables are:
  - $GLOBALS
  - $_SERVER
  - $_REQUEST
  - $_POST
  - $_GET
  - $_FILES
  - $_ENV
  - $_COOKIE
  - $_SESSION

Form Handling

- HTML forms are used to pass data to server via different input controls.
- All input controls are placed in between <form> and </form>.
- Handling forms is a multipart process. First a form is created, into which a user can enter the required details. This data is then sent to the web server, where it is interpreted, often with some error checking
- There are two pre-defined superglobals variable $_GET and $_POST are used to handle form data in php.

$_GET

- The predefined $_GET variable is used to collect data in form with method =“get” information sent from a form with the GET method is visible to everyone (it will be displayed in the browser address bar).
- The name of form field will automatically be the keys in the $_GET arrays.
- Ex: $_GET [“fname”], $_GET [“age”] etc.

$_POST:

- The predefined $_POST variable use to collect values in a form with method =“post” information sent from a form with POST method is invisible to others.
- The name of form field will automatically be the keys in the $_POST arrays.
- Ex: $_POST[“fname”], $_POST[“age”] etc.

Example (Form handled by $_GET)

//index.html

```
<html>
<head>
        <title>Form handling</title>
</head>
<body>
<form action="handle.php" method="get">
        Name: <input type="text" name="uname">
        Age: <input type="text" name="uage">
        <input type="submit" name="submit" value="submit">
</form>
</body>
</html>
```

//handle.php

```
<?php
if(isset($_GET['submit']))
{
 $n=$_GET['uname'];
 $a=$_GET['uage'];
 echo "hello $n your age is $a";
}
?>
```

Example (Form handled by $_POST)

//index.html

```
<html>
<head>
        <title>Form handling</title>
</head>
<body>
<form action="handle.php" method=“post">
        Name: <input type="text" name="uname">
        Age: <input type="text" name="uage">
        <input type="submit" name="submit" value="submit">
</form>
</body>
</html>
```

//handle.php

```
<?php
if(isset($_POST['submit']))
{
 $n=$_POST['uname'];
 $a=$_POST['uage'];
 echo "hello $n your age is $a";
}

?>
```

Form validation

- Validation is process of ensuring that form's values are correct or not.
- Form validation is process of checking that form has been filled in correctly before it is processed.
- Some type of validation are:
  - Preventing blank values.
  - Ensuring the type of values
  - Ensuring the format and range of values
  - Ensuring that values fit together.

There are mainly two methods for validating form.

Client-side validation

- Validation is done before the form is submitted
- Can lead to better user experience but not secure
- Faster and easier to do.
- client-side validation usually done using JavaScript and VBScript.
- **Server-side validation:**
- Validation is done after the form is submitted
- Server-side validation is more secure but often more tricky to code.
- it also increases load of server computer, it is slower.
- Server-side validation is done usually done using php, asp and jsp etc.

Some of Validation rules for field

Field	Validation Rules
Name	Should required letters and white-spaces
Email	Should required **@** and **.**
Website	Should required a valid URL
Radio	Must be selectable at least once
Check Box	Must be checkable at least once
Drop Down menu	Must be selectable at least once

Regular Expression

[abc]	a, b, or c
[a-z]	Any lowercase letter
[^A-Z]	Any character that is not a uppercase letter
[a-z]+	One or more lowercase letters
[0-9.-]	Any number, dot, or minus sign
^[a-zA-Z0-9_]{1,}$	Any word of at least one letter, number or _
[^A-Za-z0-9]	Any symbol (not a number or a letter)
([A-Z]{3}|[0-9]{4})	Matches three letters or four numbers

1.Checking Empty Fields using empty() function:

```
<?php
$var = "";
if( empty($var ) )
{
echo "The variable is empty";
}
else
{
echo "The variable is having some value";
}
?>
```

PHP FUNCTION preg_match()

- This function matches the value given by the user and defined in the regular expression.
- If the regular expression and the value given by the user, becomes equal, the function will return true, false otherwise.

Syntax:

Preg_match($Pattern , $Subject)

– Pattern – Pattern is used to search the string.

– Subject – input given by the user.

•If matches are found for parenthesized substrings of *pattern* and the function is called with the third argument *regs*, the matches will be stored in the elements of the array *regs*.

1.Checking matched string using preg_match() function:

```
<?php
$user= "laxmanpant2054";
$password="Laxman@2054";
if(preg_match('/2054/',$user))

{
        echo "<b>User Name is Correct</b>";
}
else
{
        echo"<div style='color:red;'>You Enter Wrong User Name</div>";
}
?>
```

2.Checking validation by using preg_match() function:

```
<?php

$a = "Laxman2054";
if(preg_match('/^[a-zA-Z0-9]{3,18}$/', $a))
{
        echo “<b>okay for login **Laxman**</b>";

}
else
{
        echo "Not valid";
}
?>
```

3.Checking validation by using preg_match() function:

```
<?php

$a = "LAXMAN2054A"; //6 character 4 digit 1 character
if(preg_match('/^[A-Z]{6}+[0-9]{4}+[A-Z]{1}$/', $a))
{
        echo "Okay**Laxman**";

}
else
{
        echo "Not valid";
}
?>
```

3.Checking validation by using preg_match() function:

```
<?php

$mail= "laxmanpant2054@gmail.com"; //making validation for email
if(preg_match('/^([a-z]+[0-9]*)@([a-z]+)\.[a-z]{2,5}$/', $mail))
{
        echo “**OKAY,It’s Valid email Address**";

}
else
{
        echo "Not valid";
}
?>
```

PHP FUNCTION filter_var()

The filter_var() function filters a variable with the specified filter

Syntax:

filter_var(*var, filtername, options*)

- Var:required. The variable to be filter.
- Filtername: optional. Specifies id or name of filter to use
- Options: optional. Specifies one or more flag/option to use.

- PHP Predefined Filter Constants
  - FILTER_VALIDATE_EMAIL :validates value as valid email
  - FILTER_VALIDATE_FLOAT: validates value as float
  - FILTER_VALIDATE_INT: validates value as integer
  - FILTER_VALIDATE_IP: validates value as IP address
  - FILTER_VALIDATE_URL: validates value as URL
  - FILTER_SANITIZE_EMAIL: removes all illegal character from email

Example:

```
<!DOCTYPE html>
<html>
<body>

<?php
// Variable to check
$email = “laxmanpant2054@gmail.com";

// Validate email
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo("$email is a valid email address");
} else {
    echo("$email is not a valid email address");
}
?>

</body>
</html>
```

Example:

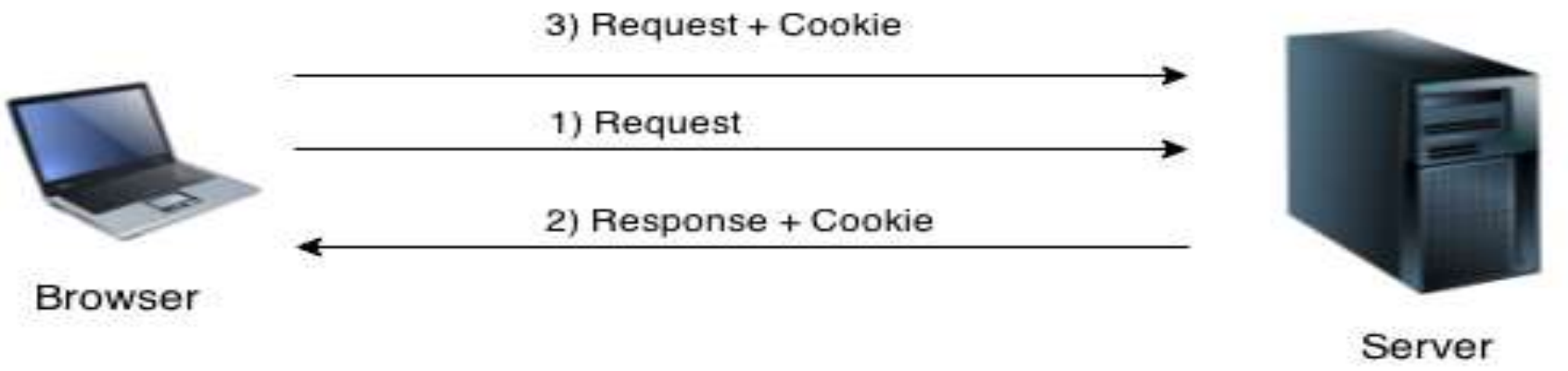
```
<!DOCTYPE html>
<html>
<head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>web</title>
</head>
<body>
<form action="conn.php" method="post">
        Full Name:<input type="text" name="n1">
        <br><br>
        Email:<input type="email" name="n2">
        <br><br>
        Password:<input type="password" name="n3">
        <br><br>
        <input type="submit" name="n4" value="Login">
</form>
</body>
</html>
```

Example: conn.php

```
<?php
if(isset($_POST['n4']))
{
		$name=$_POST['n1'];
		$email=$_POST['n2'];
		$password=$_POST['n3'];
		if(strlen($name)>=10)
		{
			echo " Max length";
		}
			if(preg_match('/^([a-z]+[0-9]*)@([a-z]+)\.[a-z]{2,5}$/', $email))

		{
			echo "The email is=$email<br>";

		}
		else{
			echo "<div style='color:red;'> Write a correct email format</div>";

		}
		if(preg_match('/^[A-Za-z0-9]{8,}+$/', $password))
		{
			echo " Valid Password is =$password ";
		}
		else
		{
			echo "Enter max 8 above</br>";
		}
}

?>
```

Introduction of cookies in php

- PHP cookie is a small piece of information which is stored at client browser. It is used to recognize the user.
- Cookie is created at server side and saved to client browser.
- Each time when client sends request to the server, cookie is embedded with request. Such way, cookie can be received at the server side.



Introduction of cookies in php cont….

- In short, cookie can be created, sent and received at server end.
- PHP setcookie() function is used to set cookie with HTTP response. Once cookie is set, we can access it by $_COOKIE superglobal variable.
- PHP $_COOKIE superglobal variable is used to get cookie.
- Implementation of cookies as given below.

Step 1: Creating Cookies

PHP **setcookie()** function is used to set cookie with HTTP response

Syntax:

setcookie(name, value, expire, path, domain, secure, httponly);

- **Name**: the name of cookie.
- **Value**: the value of cookie stored in clients computer.
- **Expire**: the time the cookies expires example time()+60 will create cookoe of lifetime 60 seconds.
- **Path:** it can be used to set the cookie path on the server. The forward slash "/" means that the cookie will be made available on the entire domain.
- **Domain**: optional, it can be used to define the cookie access hierarchy.
- **Secure**: is optional, the default is false. It is used to determine whether the cookie is sent via https if it is set to true or http if it is set to false.
- **Httponly**: optional , when true the cookie will be made accessible only through the http protocol.

Step 2.Access or view a cookie value:

- To access a cookie value, the super global variable **$_COOKIE[‘cookie_name’]** can be used.

IF we needed we must follow these steps

Step 3.Modify a cookie value:

- To modify a cookie, just set(again) the cookie using **setcookie()** function.

Stpe 4.Delete a cookie:

- To delete a cookie, use the **setcookie ()** function with an expiration date in the past

Eg:

//set the expiration date to one hour ago

setcookie (‘user’,”,time ()-3600)

Example:

Index.php

```
<?php
$cookie1_name="user";
$cookie1_value="laxman2054";
$cookie2_name="password";
$cookie2_value="Nepal@_675";
setcookie($cookie1_name,$cookie1_value,time() + (45), "/");
setcookie($cookie2_name,$cookie2_value,time() + (45), "/");
?>
```

Cookie_test.php

```
<html>
<body>
<?php
 if(isset($_COOKIE['user']) && isset($_COOKIE['password']))  {
  echo "Your  User Id  is:".$_COOKIE['user']."<br>";
  echo "Your Password is :".$_COOKIE['password']."<br>";
 }
 else
 {
   echo" Expired";
 }
  ?></body></html>
```

Session in php

- A session is a way to store information to be used across multiple pages.
- Session is used to store and pass information from one user to another , temporally (until use close the website)
- Session creates unique user id for each browser to recognize the user and avoid conflict between multiple browser.

PHP : Three Steps to Set & Get SESSION Value

Step 1 : session_start();

Step 2 : $_SESSION[name] = value; Set Session name & value

Step 3 : echo $_SESSION[name]; Get Session value

Delete Session

Step 1 : session_unset(); Remove all session variables

Step 2 : session_destroy(); Destroy the session

Initializing a session

```
session_start()
```

Before any output in php script

Accessing a session variable

```
$_SESSION [' key ']
```

Add a session value

```
$_SESSION [' key '] = value
```

Delete a session

```
session_unset()
```

Clears all session values

+

```
session_destroy()
```

Delete the session file

Example:

Index.php(session_creation)

```
<?php
 session_start();

 $_SESSION['uid']=12345;
 $_SESSION['name']=“Laxman Pant";
 ?>
```

Session_test.php

```
<?php
 session_start();

 if (isset($_SESSION['uid'])&&isset($_SESSION['name']))
                echo "wellcome, ".$_SESSION['name']."  you have successfully logged in.";
 else
              echo "you are not logged in.";
?>
```

Session_delete.php

```
<?php
  session_start();
  session_unset();
  session_destroy();
?>
```

File Handling in PHP

- File handling in PHP involves opening, reading, writing, closing and manipulating files stored on a file system. The fopen() function is used to open a file for reading, writing, or appending. The fwrite() function is used to write to a file, while the fread() function is used to read from a file.
- Using file handling access, read, write, and manipulate the file easily in php by using different predefined functions.

Modes	Description
r	**Open a file for read only**. File pointer starts at the beginning of the file
w	**Open a file for write only**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a	**Open a file for write only**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x	**Creates a new file for write only**. Returns FALSE and an error if file already exists
r+	**Open a file for read/write**. File pointer starts at the beginning of the file
w+	**Open a file for read/write**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a+	**Open a file for read/write**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist

Opening a file in PHP

fopen():- This function is used to open a file or URL. Before Reading something form file and writing to file ,we must be opened the file. The syntax of file open as follows:

fopen(filename,mode);

Example:

```
<?php
$handling=fopen("file.txt" ,"r");
if($handling)
{
        echo "File is Opened";
}
Else
{
echo “Not found";

?>
```

Reading a file in PHP

fgets():- This function is used to read a file or URL. Before Reading something form file ,we must be opened the file. For reading file in php, programmer can used the while loop , feof() ,fgetc(),fgets(),fread() functions.

feof():- This function is used to read the file until file finished.

The syntax of fread() as follows:

fgets(filename,mode);

Example:

```
<?php
$handle=fopen("handling.txt","r");

while (!feof($handle))
{

$text=fgets($handle);
echo $text."<br>";
}

?>
```

Writing to a file in PHP

The fwrite() writes to an open file. The function will stop at the end of the file (EOF) or when it reaches the specified length, whichever comes first.

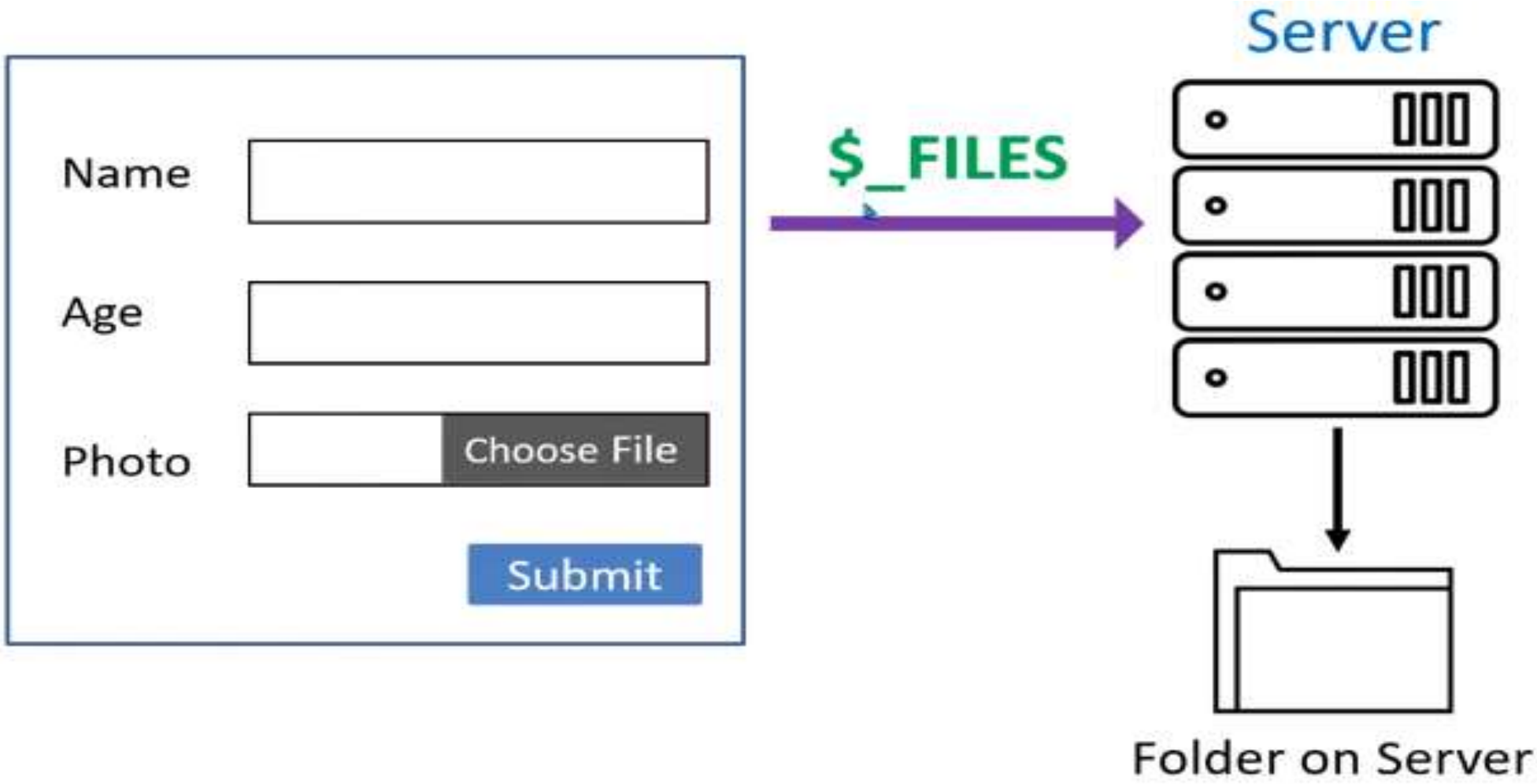
Syntax:

fwrite(file, string, length);

Example:

```
<?php
$handle=fopen("handling.txt","w");
if(fwrite($handle, “Hello hi how are you”)==FALSE)
{
echo “Could not write”
}
else
{
echo “ sucessed”
}
?>
```

File uploading in PHP



According to above figure: A PHP script can be used with a HTML form to allow users to upload files to the server by using $_FILE superglobal variable. Initially files are uploaded into a temporary directory and then relocated to a target destination by a PHP script.

The process of uploading a file follows these steps –

- The user opens the page containing a HTML form featuring a text files, a browse button and a submit button.
- The user clicks the browse button and selects a file to upload from the local PC.
- The full path to the selected file appears in the text filed then the user clicks the submit button.
- The selected file is sent to the temporary directory on the server.
- The PHP script that was specified as the form handler in the form's action attribute checks that the file has arrived and then copies the file into an intended directory.
- The PHP script confirms the success to the user.


```
<input type="file" name="image" />
```

```
$_FILES['image']
```

- name
- size
- tmp_name
- type JPG / PNG / GIF

```
move_uploaded_file(file, dest)
```

Uploading.php

```
<html>
<head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>form</title>
        <style type="text/css">
                body{border: 8px solid black;height: 50%;width: 70%;}
        </style>
</head>
<body>
        <h1>हालसालै खिचिएको फोटो अपलोड गर्नुहोस् </h1>
         <form action="uploadphp.php" method="post" enctype="multipart/form-data">
                 <input type="file" name="image"/><br><br>
                <input type="submit" value="बुझाउनुहोस"/><br>
        </form>
</body>
</html>
```

Uploadphp.php

```
<?php
if(isset($_FILES['image']))
{
        $file_name=$_FILES['image']['name'];
        $file_size=$_FILES['image']['size'];
        $file_tmp=$_FILES['image']['tmp_name'];
        $file_type=$_FILES['image']['type'];
        move_uploaded_file($file_tmp,"upload/".$file_name);
        echo " बधाइ! तपाइको फाइल अपलोड भइसकेको छ";

}
?>
```

Introduction to MySQL in PHP

- MySQL is the most popular database system used with PHP.
- MySQL is a database system used on the web
- MySQL is a database system that runs on a server
- MySQL is ideal for both small and large applications
- MySQL is very fast, reliable, and easy to use
- MySQL uses standard SQL
- MySQL compiles on a number of platforms
- MySQL is free to download and use
- MySQL is developed, distributed, and supported by Oracle Corporation
- MySQL is named after co-founder Monty Widenius's daughter: My
- The data in a MySQL database are stored in tables. A table is a collection of related data, and it consists of columns and rows.

Database and MySQL

- We need to store client data or look up data stored on the server,
- Database give us an easy way to issue **commands** to insert select, organize and remove data.

MySQL:

- Open source database , relatively easy to set up, easy to use with php.
- MySQL AB. Was a software company that was founded in 1995, where MySQL was developed.
- It was acquired by sun Microsystem in 2008
- Sun was in turn acquired by oracle corporation in 20100

Database and MySQL cont. …

- MySQL is very fast, robust, relational database management system.
- A database enables you to efficiently store search, sort and retrieve data.
- MySQL uses SQL (Structured Query Language), the standard database query language worldwide.
- MySQL is ideal for both small and large application

Connecting to MySQL

- MySQL database server can contain many databases, each of which can contain many tables.
- Connecting to MySQL via php includes following steps.
  - **Connect to MySQL**
  - **Select the database to use.**
  - **Prepare a query string.**
  - **Perform the query**
  - **Retrieve the result and handle them .**
  - **Close the connection**
- Three ways of working with PHP and MySQL:
  - MySQLi (object-oriented)
  - **MySQLi (procedural)**
  - PDO

1.Connect to MySQL:

PHP **mysqli_connect() function** is used to connect with MySQL database

Syntax:

mysqli_connect (server, username, password) or mysqli_connect (server, username, password, dbname)

Ex: $conn = new mysqli(“localhost”, “root”,” ”);

2.Select the database to use:

PHP **mysqli_select_db()** function used to select database to be used.

Syntax:

mysqli_select_db($conn, dbname);

Ex: mysqli_select_db($conn, “College”);

3.Prepare a query string.

Insert : The isert statement is used to insert data into the row of table.

Syntax:

$sql="Insert into table_name (col1, col2, … , colN) values (value1, alue2,…. , valueN)";

Or

$sql=" Insert into table_name values (value1, alue2,…. , valueN)";

Select: The select statement can be used to select data from database

Syntax:

$sql=" Select expression from table_name where conditions";

Delete: The Delete statement can be used to remove one or more row from table

Syntax:

$sql=" Delete from Table_name [where condition]";

Update: This statement is used to update or modify the value of column in the table.

Syntax:

$sql=" Update table_name set [col_name1=value,….. Column_nameN=valueN] where conditions";

4. Perform the query

To execute sql query in php, **mysqli_query()** function is used.

Syntax:

$result=mysqli_query($con,$sql);

5. Retrieve the result and handle them

To reterive and handle the result of query execution by following methods.

mysqli_num_rows(): Return the number of rows in result set.

mysqli_fetch_array() :Return the rows from the number of records available in the database as an associative array or numeric array

mysqli_fetch_row() :function returns a row from a recordset as a numeric array.

mysqli_fetch_assoc() :return the rows from the number of records available in the database as an associative array.

mysqli_fetch_object(): function returns a row from a recordset as an object.

6. Close the connection

mysqli_close() function to close MySQL connection.

Syntax:

mysqli_close($con);

- This is a very important function as it closes the connection to the database server.
- Your script will still run if you do not include this function.
- And too many open MySQL connections can cause problems for your account.
- This is a good practice to close the MySQL connection once all the queries are executed.

Example for connection to MySQL with PHP

```
<?php
 $con=mysqli_connect("localhost","root","");  //Step 1. connect to mysql.
 $dbs=mysqli_select_db($con,"college");    //Step 2. select database.

 $sql="select * from student"; //Step 3. Prepare query string.

 $result=mysqli_query($con,$sql);    //Step 4. Perform or execute query.
 echo "Roll  Name Address<br>";
if (mysqli_num_rows($result)>0) {
          while ($row=mysqli_fetch_array($result)) {
                    echo "$row[0]  $row[1]$row[2]<br>";
          }
}
mysqli_close($con);  //Step 6. close the connection.
?>
```

//Step 5. Handle Result

Example: Insert Data into database(insert.php)

```
<?php
 $con=mysqli_connect("localhost","root","");

 $dbs=mysqli_select_db($con,"college");

 $sql="insert into student values(4,'Anita','Mnr')";

 $result=mysqli_query($con,$sql);
if ($result)
     echo "Insert successfully";
else
echo "Error in Insert";
mysqli_close($con);
?>
```

Example: Delete Data From database(delete.php)

```
<?php
 $con=mysqli_connect("localhost","root","");

 $dbs=mysqli_select_db($con,"college");

 $sql="delete from student where Roll=12";

 $result=mysqli_query($con,$sql);
if ($result)
        echo "1 row deleted successfully";

else
echo "Error in delete";
mysqli_close($con);
?>
```

Example: Update Data in database(update.php)

```
<?php
 $con=mysqli_connect("localhost","root","");

 $dbs=mysqli_select_db($con,"college");

 $sql="update  student set Name='Sarita' where Roll=4";

 $result=mysqli_query($con,$sql);
if ($result)
            echo "1 row updated successfully";

else
echo "Error in update";
mysqli_close($con);
?>
```

Insert Multiple Records Into MySQL

- Multiple SQL statements must be executed with the **mysqli_multi_query()** function

Example:

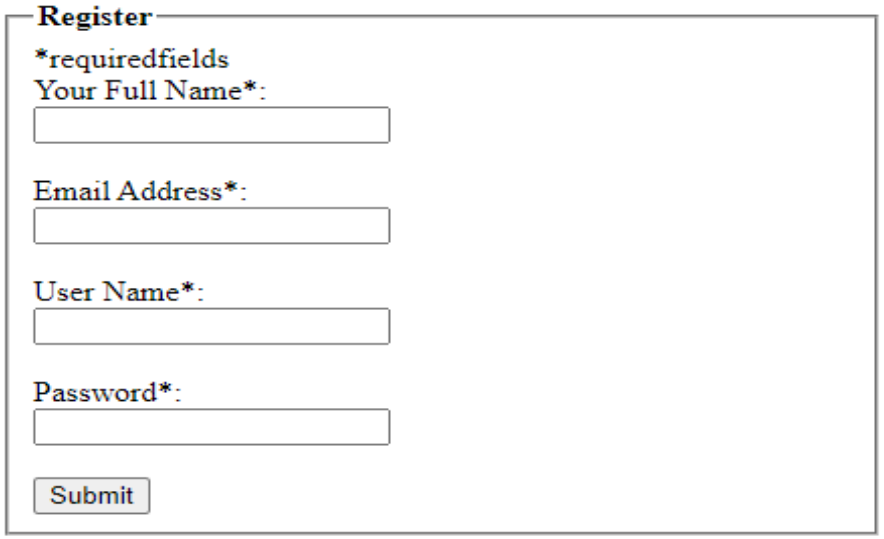
```
<?php
 $con=mysqli_connect("localhost","root","");

 $dbs=mysqli_select_db($con,"college");

 $sql="insert into student values(5,'neelam','pokhara');";
 $sql.="insert into student values(6,'Shresha','Dang');";
 $sql.="insert into student values(7,'Raj','Dadeldhura')";

 $result=mysqli_multi_query($con,$sql);
if ($result)
        echo "multiple data inserted successfully";

else
echo "Error in multiple insertion";
mysqli_close($con);
?>
```

Form Handling in MySQL

Q.N.1. Create a following given Form and make a database connection by using any scripting language.

//clientside.php

```
<!DOCTYPE html>
<html>
<head>
        <title>web</title>
        <style>
                fieldset{heigt: 50%; width: 30%;}
        </style>
</head>
<body>
        <fieldset>
                <legend><strong>Register</strong></legend>
<form action="conn.php" method="post">
```



```
*requiredfields<br>
        Your Full Name*:<br>
         <input type="text" name="n1" required>
        <br><br>
        Email Address*:<br>
        <input type="email" name="n2" required>
        <br><br>
        User Name*:<br>
        <input type="text" name="n3" required>
        <br><br>
        Password*:<br>
        <input type="text" name="n4" required><br><br>
        <input type="submit" name="n5" value="Submit">
</form>
</fieldset>
</body>
</html>
```

//conn1.php

```
<?php
if(isset($_POST['n5']))
{
        $name=$_POST['n1'];
        $email=$_POST['n2'];
        $user=$_POST['n3'];
        $password=$_POST['n4'];
        $host="localhost";
        $servername="root";
        $serverpassword="";
        $dbname="System";
        $conn=mysqli_connect($host,$servername,$serverpassword,$dbname);
        if($conn)
{
 $sql="insert into Record(Full_Name,Email,User_Name,Password)values('$name',
'$email','$user','$password')";
```

```
        $result=mysqli_query($conn,$sql);
          if($result)
                        {
                                echo "Inserted Sucessifully";
                                }

        }
else
        {
                die(mysqli_error($conn));
        }
        mysqli_close($conn);

}
```